



INFORME DEL TRABAJO

I. PORTADA

Tema:	TDA CORONAVIRUS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumnos participantes:	Camacho Monta Josue Jampier Chalco Tasna Kenneth Mateo Cunalata Mendoza Damian Alexander Tacuri Santillan Monica Sara Tisalema Guashco Darwin Joel
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DEL TRABAJO

2.1 Objetivos

General:

Diseñar, construir y comprobar el TDA Coronavirus en Java y C++, aplicando el análisis de estados, operaciones e invariantes visto en clase, y comparar el rendimiento de memoria entre ambos.

Específicos:

- Identificar claramente los estados, operaciones e invariantes del sistema.
- Diseñar el diagrama de clases en UML.
- Programar la solución completa tanto en Java como en C++.
- Analizar el uso de tipos primitivos (boolean / bool) y el impacto en memoria (JVM vs. código nativo).

2.2 Ejercicio

Desarrollar en Java un programa que simule la cadena de transmisión de un virus mediante el contacto entre personas y objetos usando:

- Un arreglo estático o de tamaño fijo.
- Al menos dos tipos de datos primitivos (boolean, int).
- Clases e instanciación de objetos.
- Comentarios que identifiquen cada concepto en el código.

2.3 Solución Propuesta

Se crea un TDA CadenaTransmision con capacidad fija para almacenar personas y objetos de la simulación. El programa está estructurado en cuatro clases principales:

Clase	Responsabilidad
Persona	Administra la identidad, el estado de infección, la contaminación de manos y las interacciones sociales/físicas.
Objeto	Representa las superficies inertes que pueden ser contaminadas mediante el estornudo de una persona infectada.
CadenaTransmision	TDA que administra los arreglos estáticos de personas y objetos.
Main	Controla la ejecución del programa, recibe datos del usuario mediante consola e instancia las interacciones.



2.4 ¿Qué es un TDA?

Un Tipo de Dato Abstracto (TDA) define:

1. Los datos que puede almacenar.
2. Las operaciones que pueden realizarse.
3. Las reglas o invariantes que siempre deben cumplirse.

En este ejercicio, el TDA es CadenaTransmision:

```
public class CadenaTransmision {  
    private static final int MAX_PERSONAS = 10;  
    private static final int MAX_OBJETOS = 10;  
  
    private Persona[] personas;  
    private Objeto[] objetos;  
    private int totalPersonas;  
    private int totalObjetos;  
}
```

Los datos están encapsulados mediante el modificador private. Para interactuar con la estructura se utilizan métodos públicos:

- cadena.registrarPersona(p);
- cadena.registrarObjeto(o);
- cadena.mostrarEstado();

2.5 Relación entre Requisitos y Código

Requisitos	Aplicación
Arreglo estático	new Persona[MAX_PERSONAS] y new Objeto[MAX_OBJETOS] declaran e instancian arreglos con capacidad fija de 10 posiciones.
Datos primitivos	Uso explícito de boolean (infectada, contaminado, manosContaminadas) e int (totalPersonas, totalObjetos).
Clase	Implementación de las clases del dominio y control: Persona, Objeto, CadenaTransmision y Main.
Instanciación	Reserva de espacio en memoria para los objetos mediante el operador new (new Persona(...), new Objeto(...)).
Herencia	No aplica: Las clases Persona y Objeto no utilizan la palabra clave extends ni derivan de una clase base.
Polimorfismo	No aplica: Cada clase define sus propios métodos de forma independiente sin sobrescribir métodos abstractos o de interfaz con @Override.
Encapsulamiento	Atributos declarados como private y acceso controlado mediante métodos públicos (getters y métodos de comportamiento).
Comentarios	Etiquetas explícitas en el código fuente (ej. // CONCEPTO: Dato Primitivo) señalando los elementos aplicados.



2.6 Pasos a seguir

Paso 1: Importar Scanner

```
import java.util.Scanner;
```

Scanner permite la entrada de datos por teclado en la consola para dinamizar los nombres de los agentes de la simulación.

Paso 2: Crear las clases de dominio

Se definen las clases que representan los elementos del dominio:

```
public class Objeto {  
    private String nombre;  
    private boolean contaminado; // CONCEPTO: Dato primitivo  
}  
  
public class Persona {  
    private String nombre;  
    private boolean infectada; // CONCEPTO: Dato primitivo  
    private boolean manosContaminadas; // CONCEPTO: Dato primitivo  
}
```

Paso 3: Encapsulamiento y construcción de objetos

Los atributos se declaran private para proteger el estado de cada objeto y se inicializan mediante constructores:

```
public Persona(String nombre, boolean infectada) {  
    this.nombre = nombre;  
    this.infectada = infectada;  
    this.manosContaminadas = false; // Invariante inicial: manos limpias  
}
```

Paso 4: Definición de operaciones del dominio

Cada clase contiene la lógica necesaria para simular la propagación del virus:

- **Persona.estornudarSobreObjeto(Objeto obj):** Contamina el objeto solo si la persona está infectada.
- **Persona.tocarObjeto(Objeto obj):** Transfiere la contaminación del objeto a las manos de la persona.
- **Persona.tenerContacto(Persona otra):** Transfiere la contaminación de manos entre personas.
- **Persona.tocarRostro():** Si las manos están contaminadas, la persona contrae la infección.
- **Persona.lavarManos():** Limpia las manos e interrumpe la cadena de transmisión.



Paso 5: Crear el Arreglo Estático

```
private static final int MAX_PERSONAS = 10;
private static final int MAX_OBJETOS = 10;

private Persona[] personas = new Persona[MAX_PERSONAS];
private Objeto[] objetos = new Objeto[MAX_OBJETOS];
```

"Estático" indica que el arreglo posee un tamaño fijo reservado en memoria al instanciarse.

Paso 6: Registrar Elementos

```
public void registrarPersona(Persona p) {
    if (totalPersonas < MAX_PERSONAS) {
        personas[totalPersonas] = p;
        totalPersonas++;
    } else {
        System.out.println("No se pueden registrar mas personas (limite alcanzado).");
    }
}
```

El objeto se almacena en la posición indicada por el contador y luego este se incrementa en una unidad.

Paso 7: Instanciar Objetos

```
Persona persona1 = new Persona(nombrePersona1, true); // Instanciación paciente cero
Persona persona2 = new Persona(nombrePersona2);       // Instanciación persona sana
Objeto objeto1 = new Objeto(nombreObjeto1);           // Instanciación objeto
```

El operador new asigna el espacio de memoria y el constructor inicializa los datos primitivos y atributos.

Paso 8: Tipos de Datos Utilizados

Tipo	Categoría	Variable	Uso
int	Primitivo	totalPersonas, totalObjetos	Contadores de capacidad ocupada en el TDA.
boolean	Primitivo	infectada, manosContaminadas, contaminado	Flags de estado del virus.
String	Referencia	nombre	Identificador del objeto o persona.
Persona[]	Arreglo / Referencia	personas[]	Vector de referencias de tipo Persona.
Objeto[]	Arreglo / Referencia	objetos[]	Vector de referencias de tipo Objeto.



Paso 9: Diagrama de Clases

```
classDiagram
class Persona {
    -String nombre
    -boolean infectada
    -boolean manosContaminadas
    +estornudarSobreObjeto(Objeto)
    +tocarObjeto(Objeto)
    +tenerContacto(Persona)
    +tocarRostro()
    +lavarManos()
    +mostrarEstado()
}
```

```
class Objeto {
    -String nombre
    -boolean contaminado
    +contaminar()
    +estaContaminado() boolean
    +mostrarEstado()
}
```

```
class CadenaTransmision {
    -Persona[] personas
    -Objeto[] objetos
    -int totalPersonas
    -int totalObjetos
    +registrarPersona(Persona)
    +registrarObjeto(Objeto)
    +mostrarEstado()
}
```

CadenaTransmision o-- Persona
CadenaTransmision o-- Objeto
Persona ..> Objeto : interactua

Paso 10: Especificación del TDA

PERSONA

- **Infectada:** indicar si la persona esta infectada
- **ManosContaminadas:** indica si tiene el virus en las manos
- Una persona puede tener las manos contaminadas sin estar infectada
- **personas:** Arreglo de capacidad fija (10) para almacenar referencias de Persona.
- **totalPersonas:** Entero que lleva el conteo de personas registradas.

OBJETO

- **contaminado:** indica si el objeto tiene presencia del virus
- **objetos:** Arreglo de capacidad fija (10) para almacenar referencias de Objeto.
- **totalObjetos:** Entero que lleva el conteo de objetos registrados.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



Operaciones:

Método	Resultado
registrarPersona(Persona p)	Almacena una persona si la capacidad no ha sido rebasada.
registrarObjeto(Objeto o)	Almacena un objeto si la capacidad no ha sido rebasada.
estornudarSobreObjeto()	Una persona infectada contamina un objeto.
tocarObjeto()	El virus pasa del objeto a las manos.
lavarManos()	Eliminar virus que interrumpe la cadena.
tenerContacto()	Una persona con las manos contaminadas transmite el virus.
TocarRostro()	Si las manos están contaminadas. La persona se infecta
mostrarEstado()	Recorre los arreglos e imprime el estado individual de cada elemento.

Invariantes:

1. $0 \leq \text{totalPersonas} \leq \text{MAX_PERSONAS}$
2. $0 \leq \text{totalObjetos} \leq \text{MAX_OBJETOS}$
3. Un objeto solo se contamina si una persona en estado infectada == true estornuda sobre él.
4. Tocar un objeto limpio no altera el estado de las manos.
5. Tener las manos contaminadas no implica estar infectado hasta que la persona ejecuta la acción tocarRostro().
6. La operación lavarManos() reinicia manosContaminadas a false.
7. La infección ocurre al tocar los ojos, nariz, boca con las manos contaminadas

Paso 11: Pseudocódigo

INICIO

Instanciar Scanner

Leer nombres de personas (p1, p2, p3) y objetos (o1, o2)

Instanciar CadenaTransmision

Instanciar Persona 1 (infectada = true)

Instanciar Persona 2 y 3 (infectadas = false)

Instanciar Objeto 1 y Objeto 2

Registrar elementos en CadenaTransmision

Mostrar Estado Inicial

Persona 1 estornuda sobre Objeto 1

Persona 2 toca Objeto 1 (Manos de P2 -> Contaminadas)

Persona 3 toca Objeto 2 (Manos de P3 -> Limpias)

Persona 2 transmite contacto a Persona 3 (Manos de P3 -> Contaminadas)

Persona 2 toca su rostro (Persona 2 -> Infectada)

Persona 3 se lava las manos y toca su rostro (Persona 3 -> Permanece Sana)

Mostrar Estado Final



FIN

Paso 12: Prueba de Escritorio

Paso	Operación	Estado Persona 1	Estado Persona 2	Estado Persona 3	Estado Objeto 1	Resultado de la prueba
1	Inicialización	Inf: SI, Manos: NO	Inf: NO, Manos: NO	Inf: NO, Manos: NO	Limpio	Se instancian los objetos en el TDA.
2	P1 estornuda en O1	Inf: SI, Manos: NO	Inf: NO, Manos: NO	Inf: NO, Manos: NO	Contaminado	El objeto 1 cambia su flag primitivo a true.
3	P2 toca O1	Inf: SI, Manos: NO	Inf: NO, Manos: SI	Inf: NO, Manos: NO	Contaminado	Las manos de P2 se contaminan.
4	P2 saluda a P3	Inf: SI, Manos: NO	Inf: NO, Manos: SI	Inf: NO, Manos: SI	Contaminado	Las manos de P3 se contaminan por transmisión.
5	P2 toca su rostro	Inf: SI, Manos: NO	Inf: SI, Manos: SI	Inf: NO, Manos: SI	Contaminado	P2 pasa a estar infectada.
6	P3 se lava las manos	Inf: SI, Manos: NO	Inf: SI, Manos: SI	Inf: NO, Manos: NO	Contaminado	Se rompe la cadena de transmisión en P3.

2.7 Análisis de resultados

Implementación del TDA con datos primitivos

El TDA CadenaTransmision y sus clases del dominio (Persona y Objeto) basan toda la lógica del contagio en variables de tipo primitivo:

- Clase Persona: Usa boolean (Java) y bool (C++) en las variables infectada y manosContaminadas para controlar el estado de salud y el transporte del virus.
- Clase Objeto: Emplea el tipo boolean / bool en la variable contaminado para marcar si la superficie tiene carga viral.
- TDA CadenaTransmision: Utiliza int en totalPersonas y totalObjetos para controlar el número de elementos registrados dentro de los arreglos estáticos.



Comparación de memoria y opción más eficiente

Veredicto: C++ es la mejor opción en términos de consumo de memoria.

Justificación:

1. **Sin sobrecarga de máquina virtual:** Java añade de 8 a 16 bytes adicionales por cada objeto instanciado debido a los encabezados que exige la JVM [1]. C++ no tiene este consumo extra y solo ocupa el tamaño de los datos declarados.
2. **Uso del Stack:** En C++ los objetos de la simulación se crearon en la pila (stack), lo que hace que se liberen de inmediato al terminar la función sin saturar la memoria. En Java todo pasa obligatoriamente al heap.
3. **Control directo de recursos:** Java depende del Garbage Collector para limpiar memoria, lo que genera un gasto de recursos en segundo plano [2]. C++ maneja los punteros y variables de forma directa sin esa carga.

2.8 Conclusiones

- El TDA logró de manera adecuada la abstracción del recorrido del virus desde la superficie contaminada hasta el rostro de un individuo.
- La utilización de tipos primitivos disminuyó la carga de datos al gestionar estados en el interior de la clase.
- C++ fue más eficaz en términos de memoria que Java, ya que logró eludir el Garbage Collector y el overhead de la máquina virtual (JVM).

2.9 Referencias Bibliográficas

- [1] Baeldung, "Reduce Object Header Size and Save Memory in Java 25," *Baeldung*, may 2026. [En línea]. Disponible: <https://www.baeldung.com/java-object-header-reduced-size-save-memory>. [Accedido: 26-ago-2026].
- [2] "The Cost of Garbage Collection for State Machine Replication," *arXiv preprint*, arXiv:2405.11182, 2024. [En línea]. Disponible: <https://arxiv.org/pdf/2405.11182>. [Accedido: 26-ago-2026].



2.10 Anexos

Diagramas UML

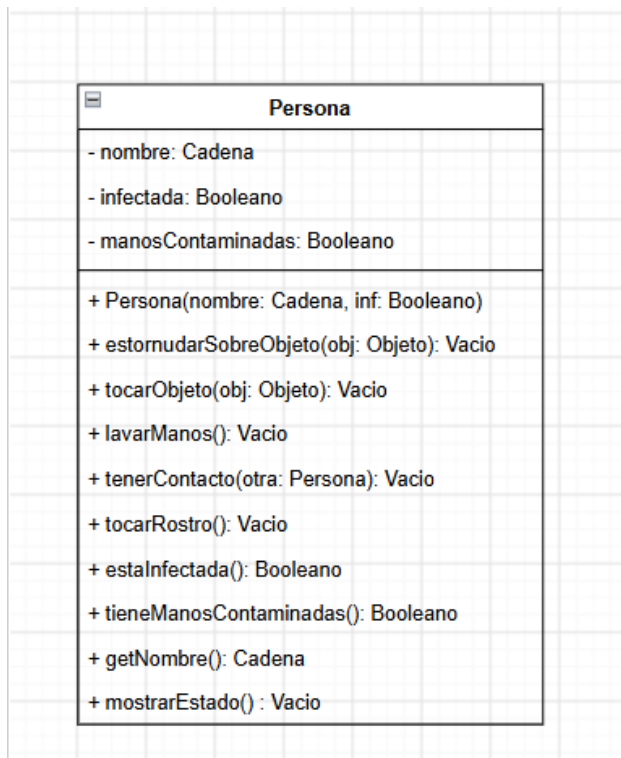


Figura 1. Diagrama de la clase persona.

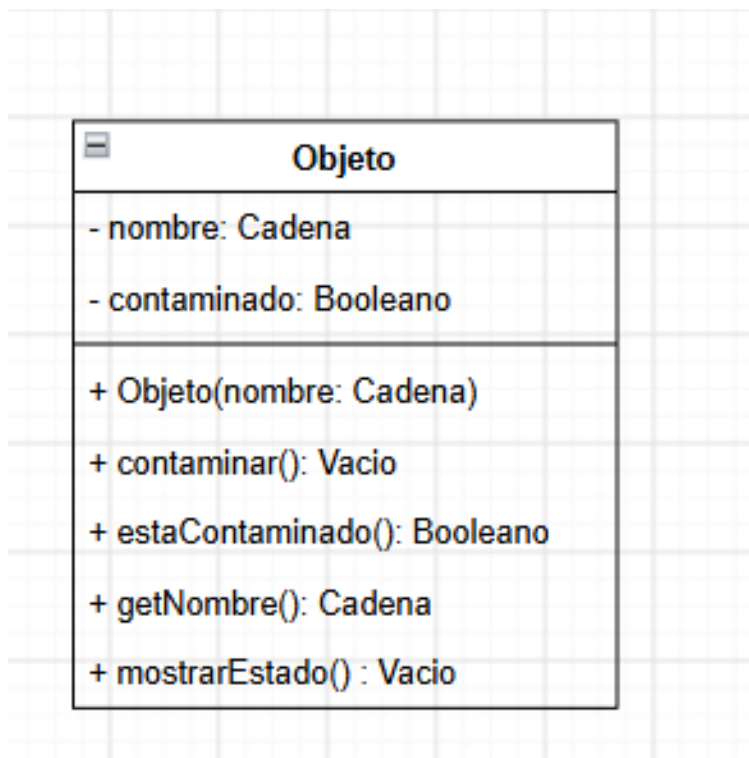


Figura 2. Diagrama clase objeto

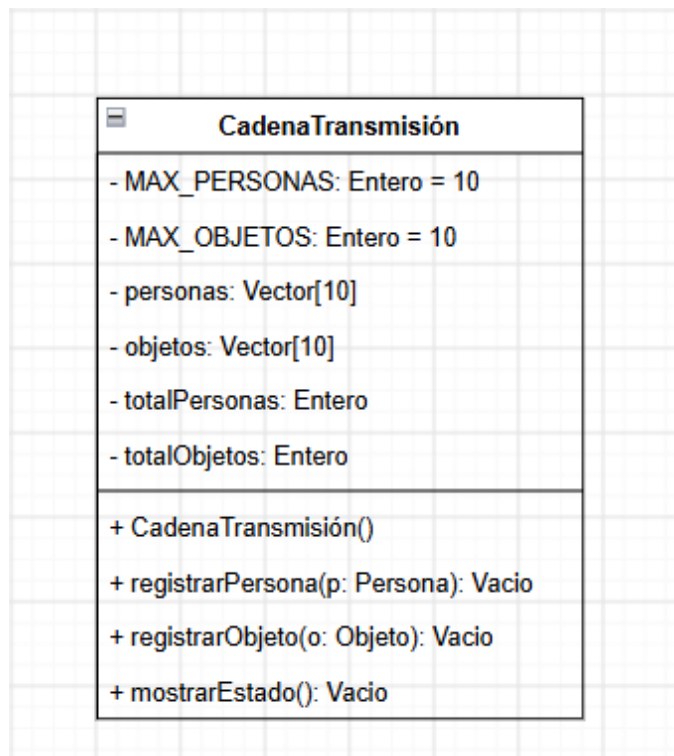


Figura 3. Diagrama de la clase *CadenaTransmision*

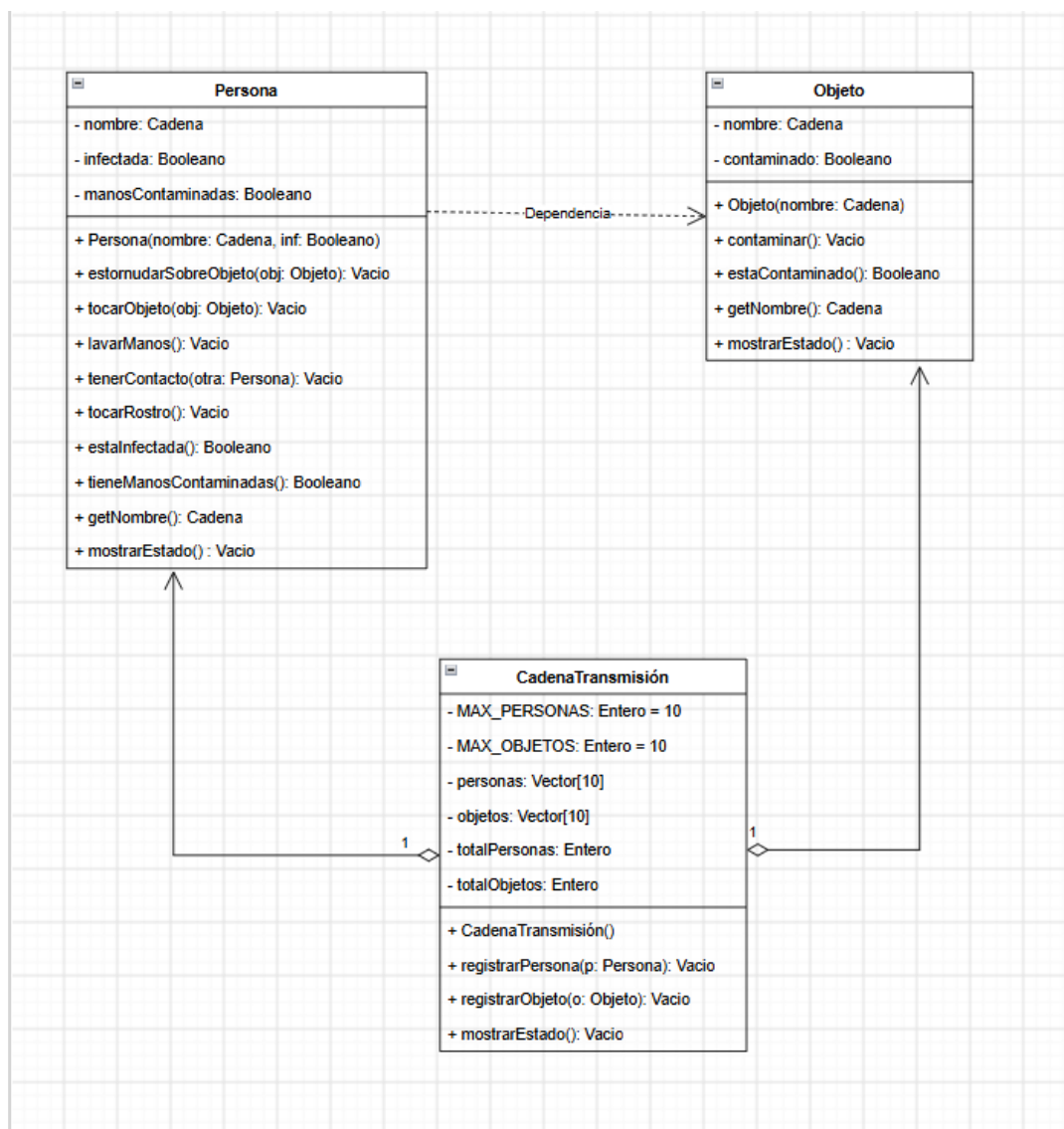


Figura 4. Diagrama de flujo del TDA Coronavirus



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO 2026 – DICIEMBRE 2026



```
===== ESTADO DE LA CADENA DE TRANSMISION =====
-- Personas --
Persona [Alex] -> Infectada: SI | Manos contaminadas: NO
Persona [Fer] -> Infectada: NO | Manos contaminadas: SI
Persona [Mayra] -> Infectada: NO | Manos contaminadas: NO
-- Objetos --
Objeto [Mause] -> CONTAMINADO
Objeto [Mesa] -> limpio

=====

Fer tiene contacto con Mayra -> le contamina las manos.

¿La persona 2 esta infectada antes de tocarse el rostro? NO
Fer se toca el rostro con las manos contaminadas -> SE INFECTA.
Mayra se lava las manos -> manos limpias, cadena de transmision interrumpida.
Mayra se toca el rostro con las manos limpias -> no ocurre infeccion.

ESTADO FINAL

===== ESTADO DE LA CADENA DE TRANSMISION =====
-- Personas --
Persona [Alex] -> Infectada: SI | Manos contaminadas: NO
Persona [Fer] -> Infectada: SI | Manos contaminadas: SI
Persona [Mayra] -> Infectada: NO | Manos contaminadas: NO
-- Objetos --
Objeto [Mause] -> CONTAMINADO
Objeto [Mesa] -> limpio

=====

Presione una tecla para continuar . . .
```

Figura 5. Ejecución del programa en C++.

```
Nombre del objeto 1 (sera contaminado por la persona 1): Raton
Nombre del objeto 2 (permanecera limpio): Mesa
=====

ESTADO INICIAL

===== ESTADO DE LA CADENA DE TRANSMISION =====
-- Personas --
Persona [Luisa] -> Infectada: SI | Manos contaminadas: NO
Persona [Mirian] -> Infectada: NO | Manos contaminadas: NO
Persona [Alex] -> Infectada: NO | Manos contaminadas: NO
-- Objetos --
Objeto [Raton] -> limpio
Objeto [Mesa] -> limpio

=====

Luisa estornuda sobre Raton -> objeto CONTAMINADO.
Mirian toca Raton -> sus manos quedan CONTAMINADAS.
Alex toca Mesa (limpio) -> sus manos siguen limpias.

ESTADO TRAS EL CONTACTO CON LOS OBJETOS

===== ESTADO DE LA CADENA DE TRANSMISION =====
-- Personas --
Persona [Luisa] -> Infectada: SI | Manos contaminadas: NO
Persona [Mirian] -> Infectada: NO | Manos contaminadas: SI
Persona [Alex] -> Infectada: NO | Manos contaminadas: NO
-- Objetos --
Objeto [Raton] -> CONTAMINADO
Objeto [Mesa] -> limpio

=====
```

Figura 6. Ejecución del programa en Java.

A continuación, se adjunta el enlace del repositorio donde se podrá visualizar los códigos realizados en C++ y en Java, además de las fotografías tomadas de ejecución junto con el diagrama UML y el presente documento en formato .docs y .pdf.

https://github.com/Damieng7w/Estructura_Datos